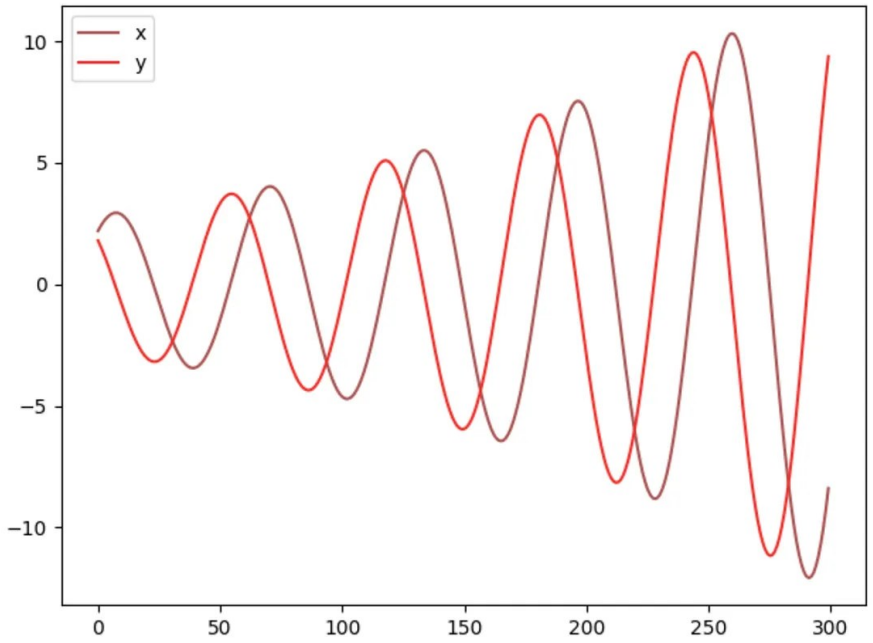
NEED FOR DIFFUSION MODELS

- GANs produce sharp images but are hard to train, using a minimax objective
- There is no explicit likelihood or density estimation
- As a result, we face issues like mode collapse, training instability and sensitivity to hyperparameters



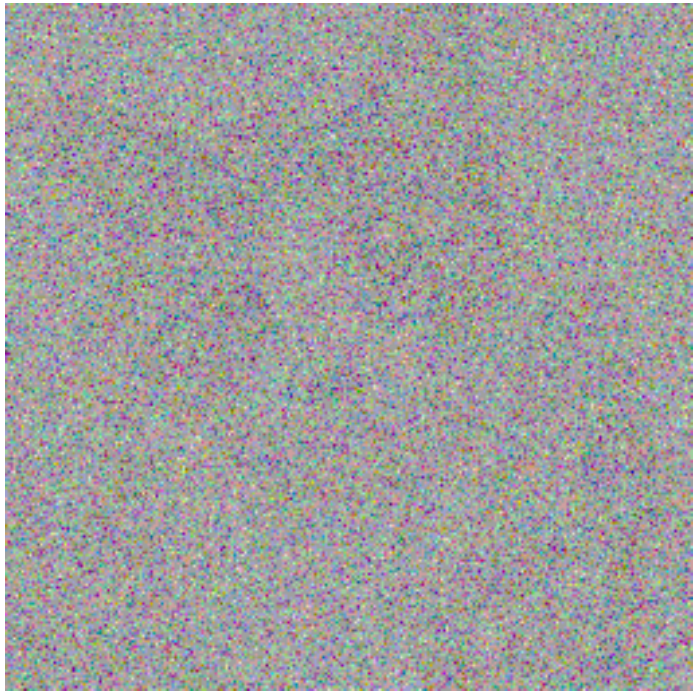
Can generation be framed as a *gradual*, *stable*, *reversible process*?

Instead of learning to fool a discriminator, can we learn to construct data step by step?

Diffusion models answer this by modelling **how data is destroyed and rebuilt**

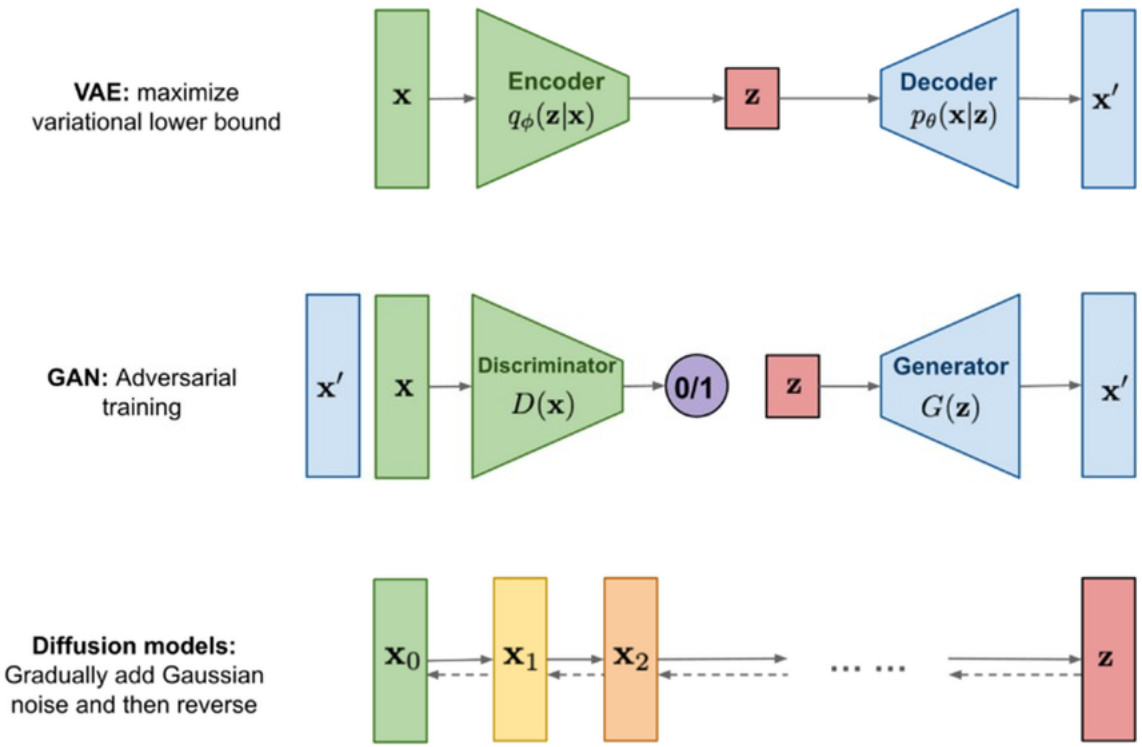
DIFFUSION MODELS

- Diffusion models are a class of generative models that learn to create data by reversing a process of gradual destruction
- Instead of trying to map noise directly to an image (like a GAN), a diffusion model learns to "denoise" an image one tiny step at a time
- Basically, they are used to transform a field of pure Gaussian noise into a high-fidelity, structured image



WHAT DIFFUSION MODELS OPTIMISE?

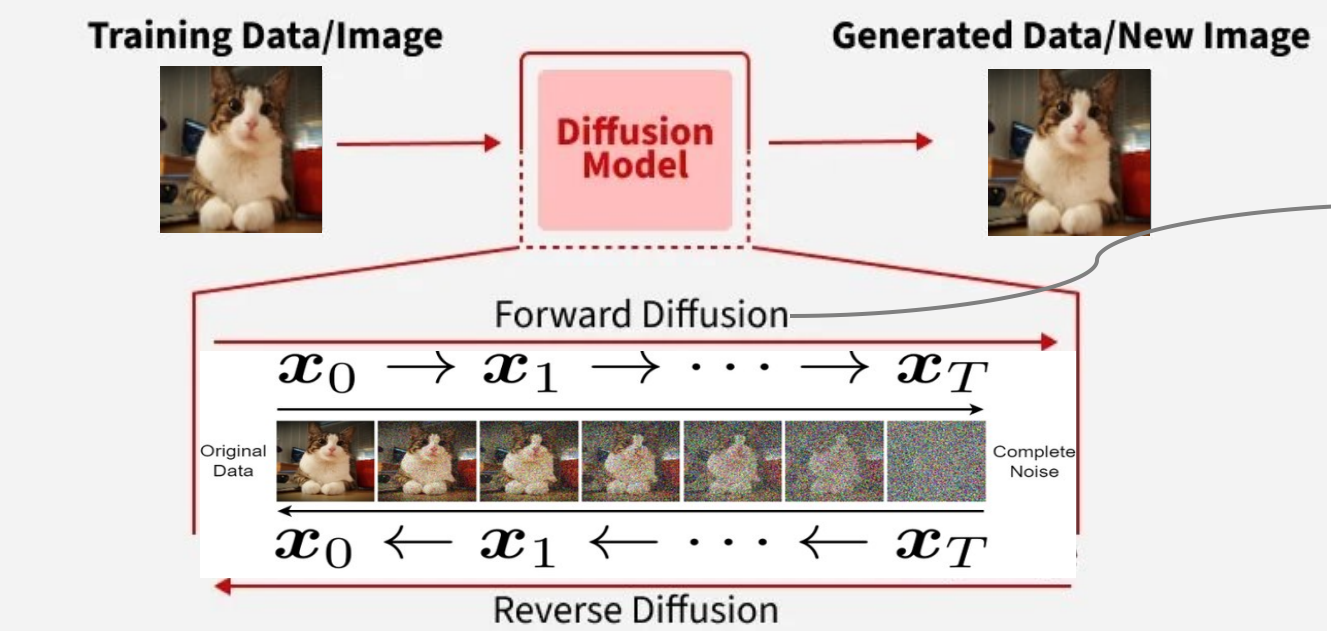
- Diffusion models are likelihood-based generative models
- They explicitly model the data distribution
- Training is stable and does not rely on adversarial competition
- Diversity is encouraged by design, not by heuristics
- Mathematically, diffusion models try to gradually convert one distribution into another using a Markov chain



NOTE: *We will focus on a specific diffusion model –* ***Denoising Diffusion Probabilistic Models (DDPM)***
These are often used interchangeably as DDPM is the original instantiation of diffusion models
DDPM specifies how noise is added, how the reverse process is learned, and how training is done

WORKING INTUITION

The working of a diffusion model is split into two distinct processes:
the **forward diffusion** and the **reverse diffusion**

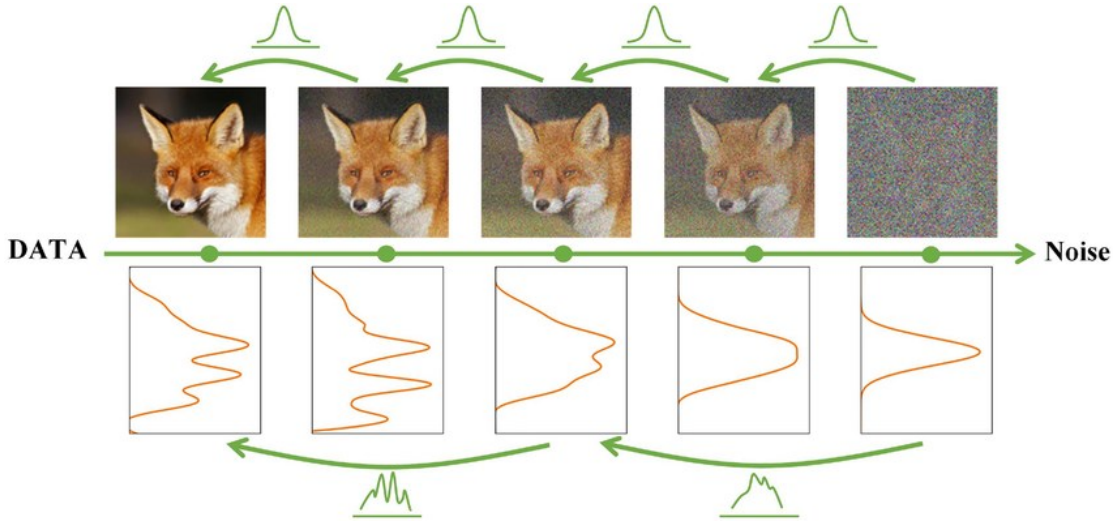


- *Start with a real image*
- *Add a small amount of Gaussian noise*
- *Repeat over many steps*
- *Eventually, the image becomes pure noise*
- *This process is **fixed and non-learned***

- *If we can destroy data step by step…*
  can we learn to reverse this destruction?
- *Generation = removing noise iteratively*
- *Each step slightly improves image quality*

ADDING GRADUAL NOISE

- We add noise *gradually* through the forward diffusion process
- Each step removes a small, controlled amount of information
- This mirrors encoder behaviour, where structure is broken progressively
- How information is destroyed defines how it can be reconstructed
- Gradual corruption avoids sudden, irreversible information loss and enables smooth and stable reverse (denoising) process
- Transforms generation into many simple, learnable steps



APPLICATIONS AND USE CASES

- Diffusion models are not limited to images; they define a general strategy for learning complex data distributions
- Generation is framed as iterative refinement rather than one-shot decoding
- This flexibility enables broad applications across domains
- Diffusion models are applicable wherever data can be incrementally corrupted and restored, some examples are
  - High-quality image generation and editing
  - Denoising, super-resolution, and inpainting tasks
  - Text-to-image and image-to-image generation
- It can be extended naturally to audio, video, and 3D data: speech synthesis, music generation, and motion modelling
- It can also be applied in scientific domains such as medical imaging and molecule generation

COMPARISON

Feature	Variational Autoencoders (VAEs)	Generative Adversarial Networks (GANs)	Diffusion Models
Training Stability	**High** Uses a stable, likelihood-based objective function	**Low** Suffers from "Mode Collapse" and unstable adversarial oscillations	**High** Uses a predictable, step-by-step denoising objective
Output Quality	**Moderate** Often results in "blurry" images due to pixel averaging	**Very High** Produces extremely sharp, photorealistic details	**Very High** Currently the state-of-the-art for high-fidelity synthesis
Controllability	**High (Latent)** Smooth latent space allows for easy feature interpolation	**Moderate** Requires complex "latent space walking" or conditional labels	**Very High (Semantic)** Excellent alignment with text prompts and masks
Diversity	**High** Good coverage of the entire data distribution	**Low to Moderate** Often repeats certain patterns (Mode Collapse)	**High** Captures complex distributions with very high variety
Inference Speed	**Fast** Requires only one pass through the decoder network	**Fast** Requires only one forward pass through the generator	**Slow** Requires hundreds of iterative "denoising" steps
Primary Limitation	Mathematical constraints lead to loss of fine detail	Extremely difficult to tune; "fragile" training process	Massive computational cost for sampling/generation

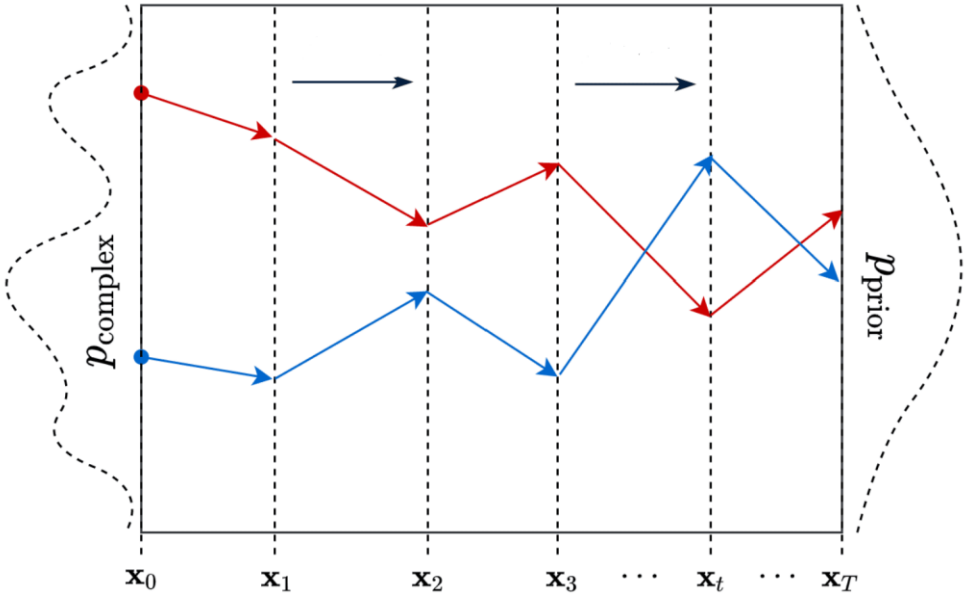
FORWARD DIFFUSION

- Forward diffusion is a probabilistic process
- Diffusion defines a forward noising process – it progressively destroys structure in the data over time
- We add noise to corrupt the images such that they move "away" from their existing subspace
- It is modelled as a **Markov chain**

Mean μ_t — Variance Σ_t

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t \boldsymbol{I}\right)$$

  - q denotes a fixed, **known** process from step x_{t-1} to x_t
  - $\beta_t \in (0,1)$ is a small, predefined variance, controlling noise
  - $\sqrt{1-\beta_t}$ is scaling down the signal from previous step
  - Covariance $\beta_t \boldsymbol{I}$: Gaussian noise with variance β_t added in each step



At the end, the complex data distribution is wholly transformed into a chosen "simple" distribution

For a large $t = \mathrm{T}$:

$$x_{\mathrm{T}} = \mathcal{N}(0, \boldsymbol{I})$$

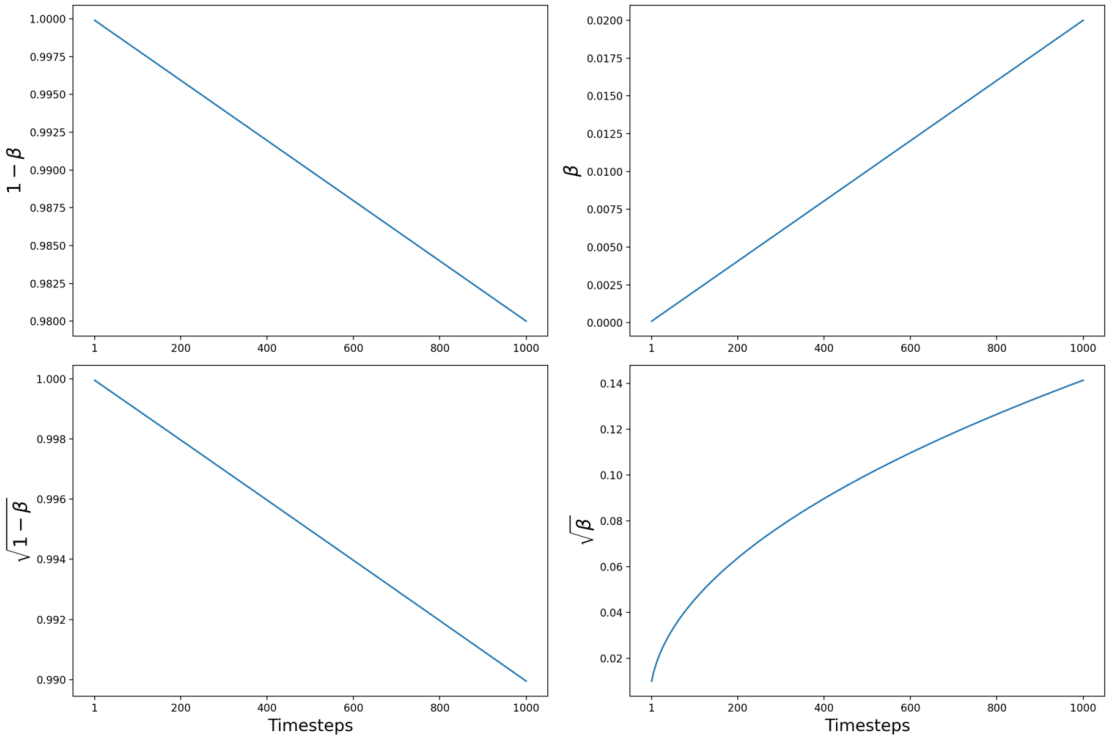
MOVING THROUGH STEPS

- To sample x_t

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon; \qquad \varepsilon \sim \mathcal{N}(0, I)$$

- Here, the ε is the noise term that is randomly sampled from the standard gaussian distribution
- In this way, starting from x_0, the original image is iteratively corrupted from $t = 1 \ldots \mathrm{T}$
- Basically, we scale down the data with each forward step (by a $\sqrt{1-\beta_t}$ factor) so that variance does not grow when adding noise
- **There is a problem**: whenever we need a latent sample x at timestep t, we must perform $t-1$ steps in the Markov chain

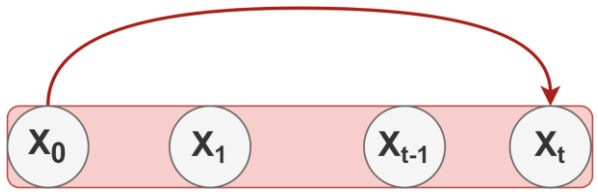
The rate at which this noise is added is not arbitrary but is governed by a scheduler



variance schedule vs timesteps

NOISE FORMULATION





- Expanding recursively, we can write the closed form forward distribution

$$q(x_t \mid x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t}\, x_0, (1-\bar{\alpha}_t)\boldsymbol{I}\right)$$

  - $\alpha_t = 1 - \beta_t$; signal retained at step t
  - $\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$; cumulative signal retained after t steps
- Rewriting the forward process,

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon; \quad \varepsilon \sim \mathcal{N}(0, I)$$

- This allows sampling any x_t directly from x_0 without needing to simulate the full chain during training

But we want a step-by-step destruction, so why use the closed-form noise?

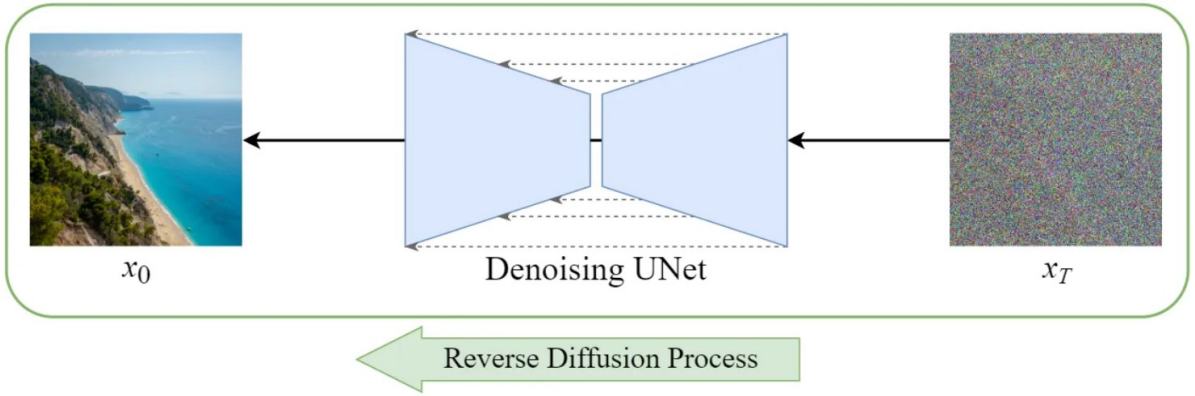
- *The gradual process defines what step "t" means in terms of information loss*
- *The closed form allows us to sample at step "t" without the need for the whole calculation*
- *This direct sampling is only a computational shortcut, not a conceptual one*

REVERSE DIFFUSION - DENOISING

- The reverse process is unknown:

$$p_\theta(x_{t-1} \mid x_t)$$

- Unlike the forward process, it is **learned** and uses a neural network, usually a denoising **U-Net**
- Generation = sampling from this reverse chain to create a slightly “cleaner” version of the image in each step



- Reverse transitions are modelled as Gaussians

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_t)$$

  - θ: learnable parameters
  - Mean is predicted by a neural network
  - Variance is often fixed or scheduled
- The reverse process is Gaussian in form but unknown in parameters
- The mean depends on the data distribution, which is not analytically available
- Given a noisy sample, many predecessors are possible
- Choosing the correct direction requires knowledge of the data manifold, a NN learns this data-dependent denoising

WHAT IS PREDICTED?

- Input to the U-Net is the image x_t (which contains the noise pattern mixed with signal) together with the timestep t
- It processes x_t like any image and outputs an image-shaped noise estimate
- Instead of predicting μ directly, the model predicts noise for a step:

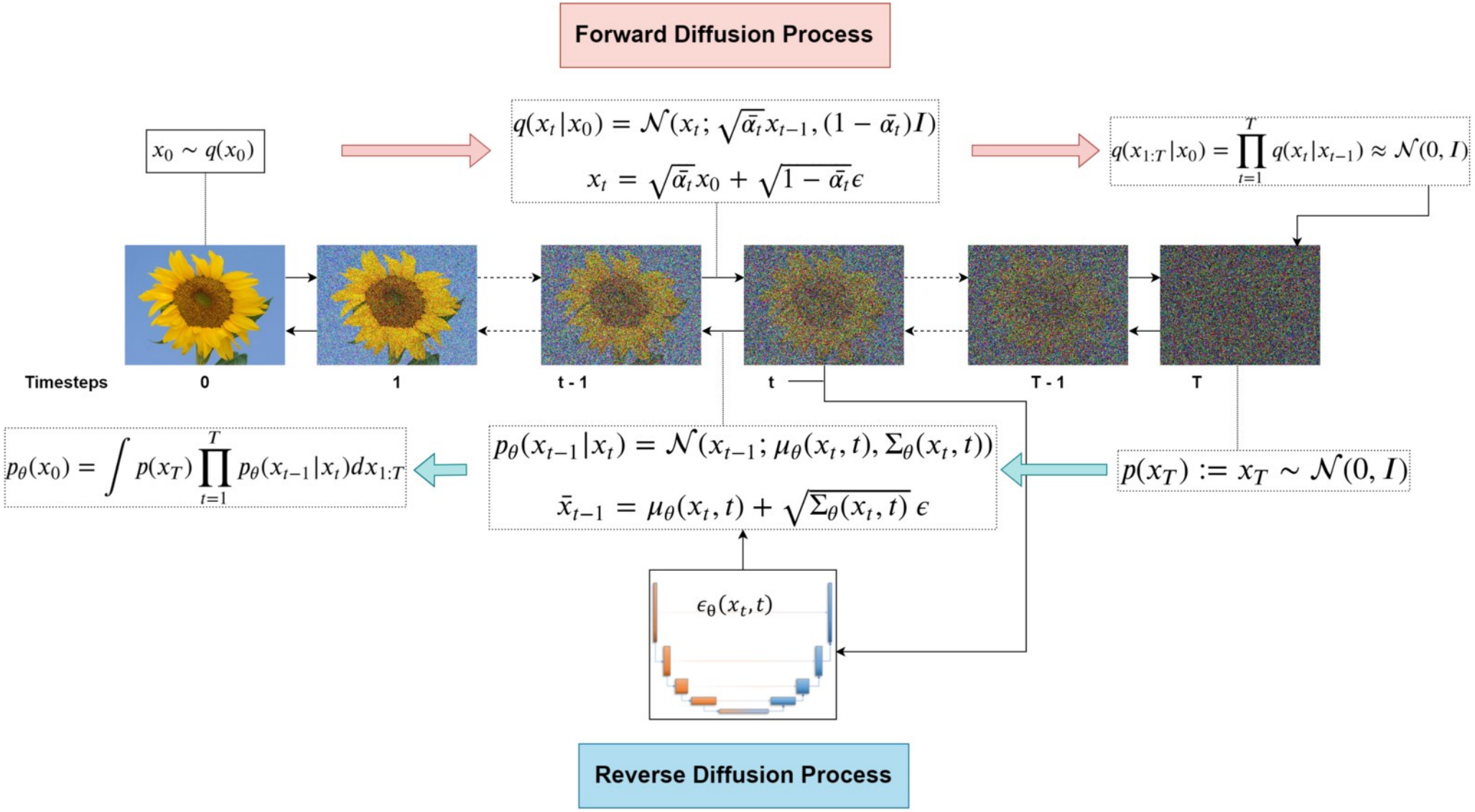
$$\varepsilon_\theta(x_t, t)$$

- Then using this,

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right)$$

- More generally, we remove a small amount of noise to go from x_t to x_{t-1}, controlling the amount of noise removed in each step with β

PROCESS OVERVIEW

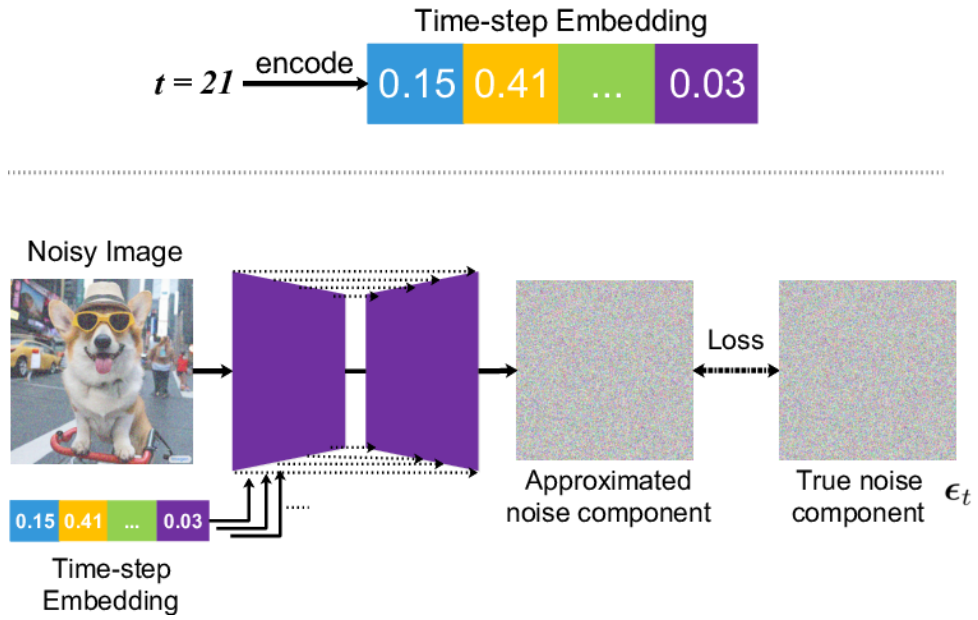


TIME CONDITIONING

- In diffusion models, the same underlying image appears very different at different timesteps due to varying noise levels
- The network receives only the noisy image x_t; from appearance alone, it cannot reliably infer how noisy the input is
- Therefore, the timestep t must be provided explicitly as an additional input
- The model is trained to predict noise as a function of both inputs:

$$\varepsilon_\theta(x_t, t)$$

- The timestep t is converted into a vector embedding and injected into the U-Net at multiple layers
- This allows the network to adapt its denoising behaviour according to the current noise regime

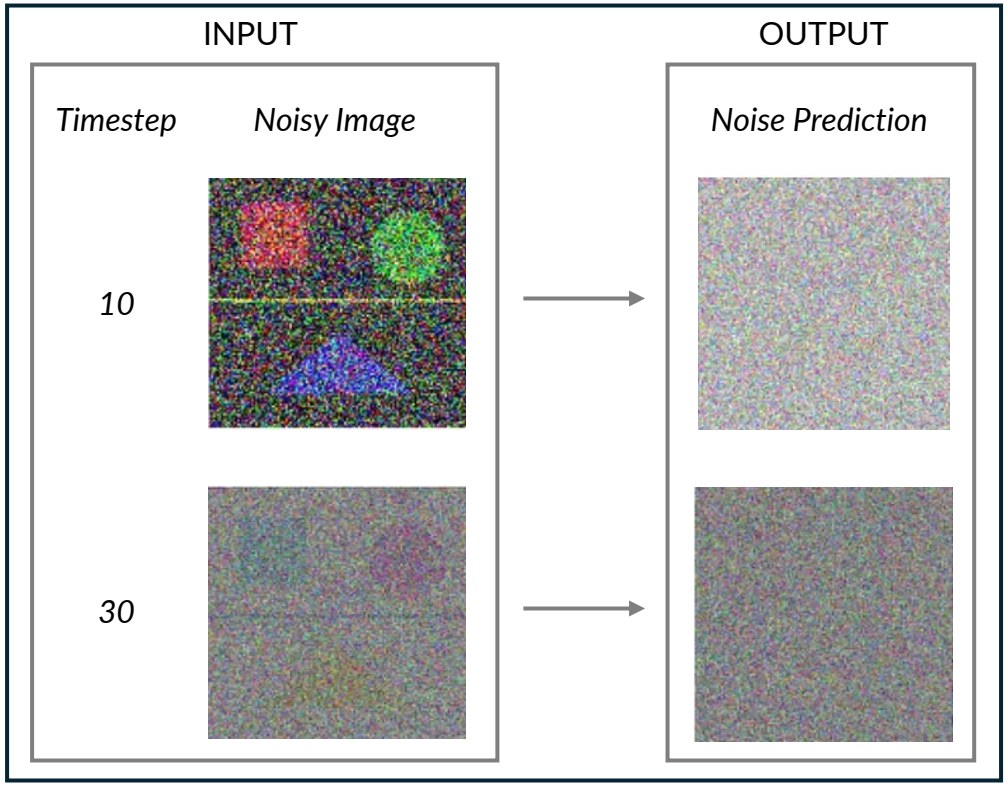


WHY TIME CONDITIONING MATTERS?

- A single diffusion model is used for all timesteps, often hundreds or thousands
- Time conditioning tells a diffusion model how much noise to remove at a step
- Without time conditioning:
  - The network would mix denoising behaviours across steps
  - Training would become unstable or converge to an averaged solution
- Timestep embeddings act as a control signal, telling the network
  - how strong the noise is
  - how aggressive the denoising should be
- One network behaves like many specialised denoisers, with t selecting the appropriate behaviour at each step

DDPM TRAINING SETUP

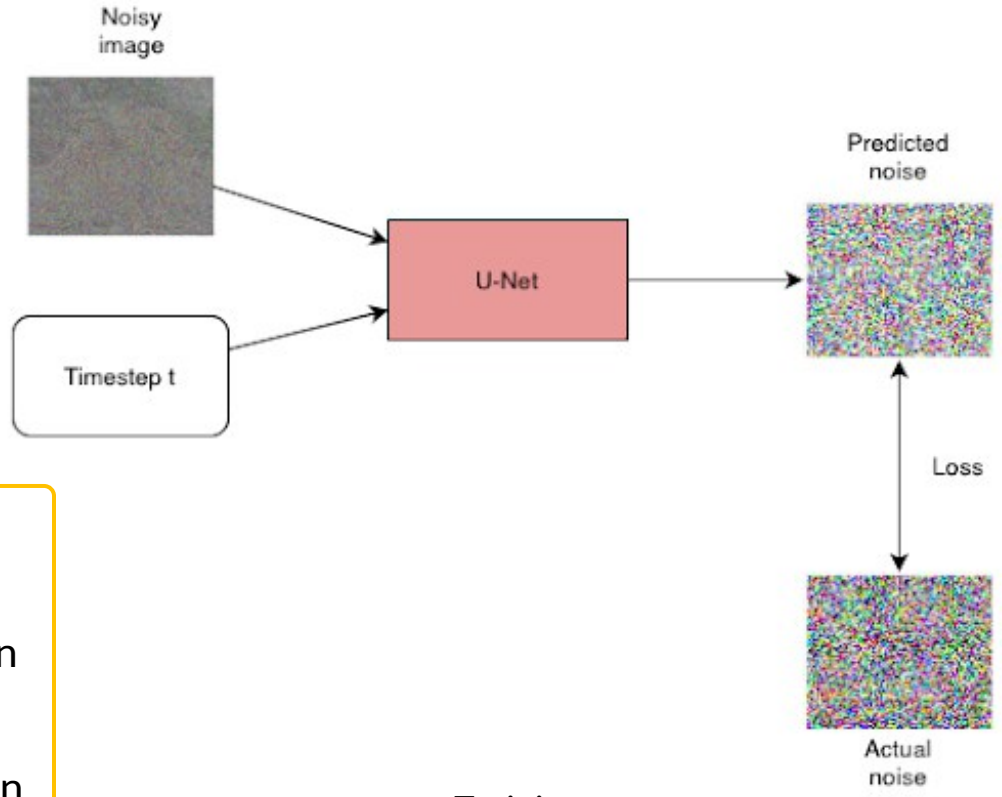
- Forward diffusion process $q(x_t|x_{t-1})$ is fixed
- Noise schedule β_t is predefined → corruption is deterministic
- Only the reverse denoising model (reconstruction) is learned
- Image generation (reverse diffusion) relies on the U-Net's ability to predict the noise present in an image at a timestep
- The network predicts the noise in the image using the timestep and noise information provided by the scheduler
- For training, a dataset with images of different levels of noise and associated timesteps is used, allowing the network to refine noise prediction over time
- As the training progresses, the network becomes more adept at distinguishing and predicting the noise patterns



Sample Training Dataset

TRAINING LOOP

1. Sample real image x_0
2. Sample timestep $t \sim \text{Uniform}(1, T)$
3. Sample noise $\varepsilon \sim \mathcal{N}(0, I)$
4. Construct noisy image x_t
5. Predict $\hat{\varepsilon} = \varepsilon_\theta(x_t, t)$
6. Minimise denoising loss



Training

- This is repeated independently for every batch
- The loss in the training process is defined by the difference between the predicted noise and the actual noise added to the image
- This difference is measured directly at the sample level, not between probability distributions
- The loss metric* used is MSE (L2 loss) on the noise prediction
- Diffusion training is stable due to this well-behaved regression loss – no adversarial interaction or discriminator collapse like GAN

**KL-divergence (recall VAE) appears in the theoretical variational derivation, but simplifies to MSE in practice*

TRAINING OBJECTIVE

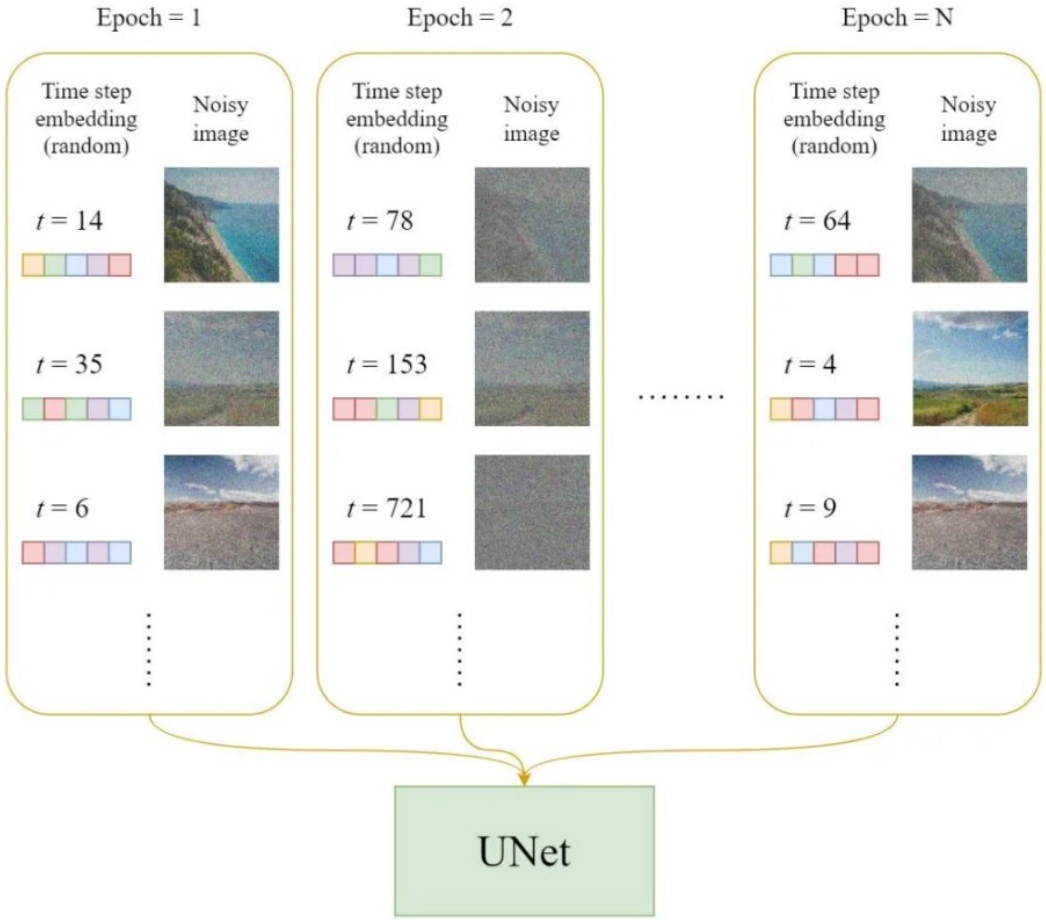
- Training reduces to a denoising regression task
- For each training step, sample:
  - real image x_0
  - timestep t
  - noise ε
- Corrupt $x_0 \rightarrow x_t$
- Predict ε from (x_t, t)
- The objective function here is

$$\mathcal{L} = \mathbb{E}_{x_0,\varepsilon,t}[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2]$$

- This is a simple Mean Squared Error loss
- Corresponds to maximising a variational lower bound on likelihood
- Why L2 Loss?
  - The injected noise ε follows a Gaussian distribution
  - Minimising L2 is equivalent to maximum likelihood estimation
  - The loss is stable, well-scaled, and convex for each timestep

ARCHITECTURE CHOICES

- The model must understand global semantics (object identity) and preserve local structure (edges, textures)
- It also needs to operate at multiple noise levels
- U-Net naturally matches the denoising task:
  - coarse-to-fine reconstruction
  - spatial alignment across noise levels
- Encoder captures global context, decoder reconstructs spatial detail, and skip connections preserve fine structure
- Early layers mostly see noise while later layers refine the structure
- Skip connections stabilise the gradients and preserve spatial information
- So, denoising is spatial also, and not semantic-only

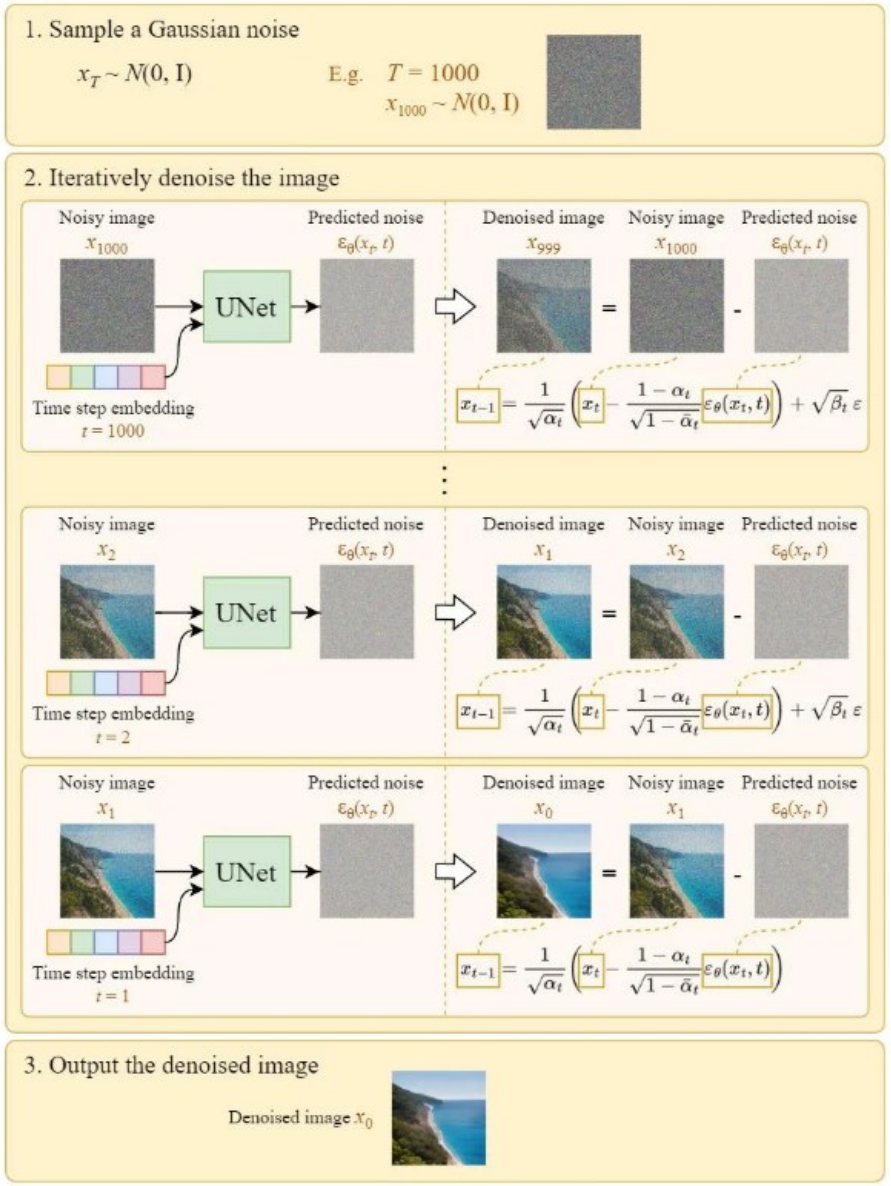


GENERATIVE PROCESS AND SAMPLING

- Start from pure noise: $x_T \sim \mathcal{N}(0, I)$
- Iteratively apply learned reverse transitions:

$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0$$

- Each step removes a small amount of noise
- However, sampling is sequential and slow, around T=1000 steps
- Each step requires a neural network forward pass
- This gives high fidelity but slow inference

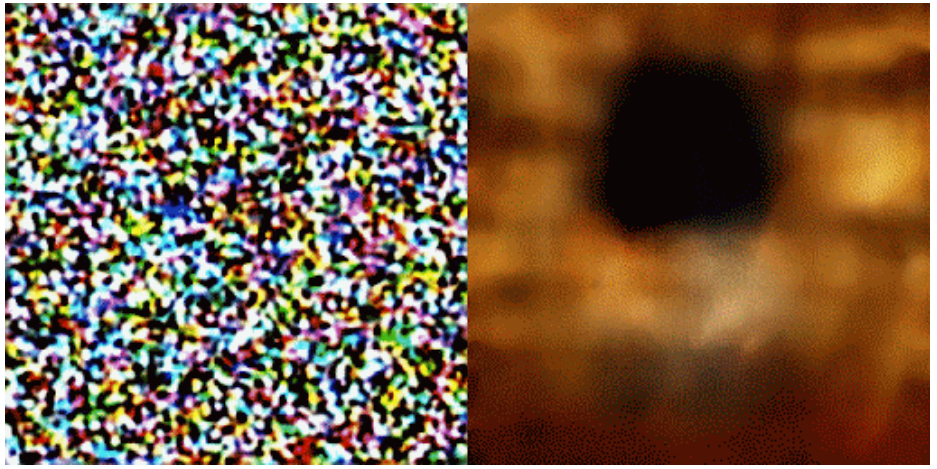


SAMPLING WITH FEWER TIMESTEPS

- Training uses many timesteps (e.g. 1000), which teaches the model fine-grained denoising
- Sampling does not need to follow every step; we can skip steps during generation
- For example, train with 1000 steps but sample with 50 steps

$$x_{1000} \rightarrow x_{980} \rightarrow x_{960} \rightarrow \cdots$$

- At each sampling step x, the scheduler gives:
  1. **Prediction of** x_{t-1}: the exact amount of noise needed to move from our current prediction to the previous
  2. (Final) **Prediction of original** x_0: remove all noise at once
- Even after this, the image quality does not collapse
  - The model has learned denoising at all noise levels
  - Skipping steps still lands in familiar regions
  - Small loss in fidelity, large gain in speed



1 2

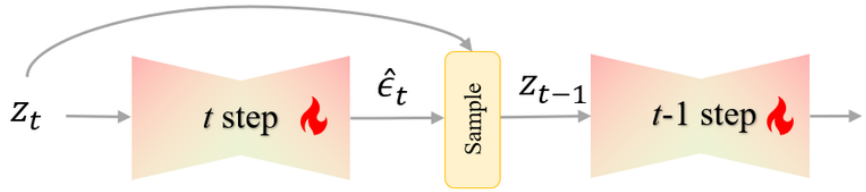
Step-by-step evolution of the two samples given by the scheduler over the complete reverse diffusion process

IMPROVING SAMPLING WITHOUT RETRAINING

- DDPM sampling is sequential and requires hundreds or thousands of steps
- **Denoising Diffusion Implicit Models (DDIM)** is not a new diffusion model, but a modification of the sampling process
- The trained DDPM model is reused; no retraining is required

How it is different from just sampling with fewer timesteps?

- DDPM generation is stochastic; naïvely skipping steps can be unstable
- DDIM removes randomness in the reverse process, making sampling deterministic
- Each step is computed directly using the predicted noise (or x_0)
- Deterministic sampling enables reproducibility
- So, we have fewer steps, faster generation, at the cost of slightly reduced diversity (in fully deterministic DDIM)

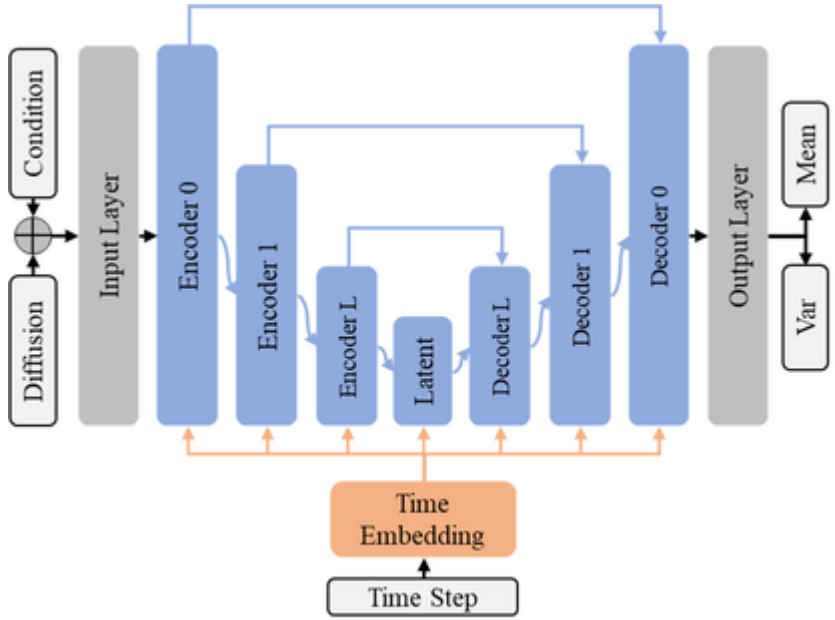


$$x_T \rightarrow x_{T-k} \rightarrow x_{T-2k} \rightarrow \cdots \rightarrow x_0$$

- *Fewer, larger deterministic jumps*
- *Reverse path is redefined so skipping is valid*

CONDITIONAL DIFFUSION

- Just like standard GANs, unconditional diffusion models generate realistic images but have no control over content
- Real world applications require control over generated class, attributes or text
- **Conditional diffusion models** guide the image generation using external information, rather than producing outputs from pure random noise
- Like cGANs, conditioning information is provided during training
- The model learns $p(x_0 \mid c)$ where c is a condition like class label, text embedding or an image or feature vector
- In cGANs, the discriminator is required only during training
- In diffusion models, the conditioning signal remains active throughout inference – it enters the denoising network directly



(a) UNet

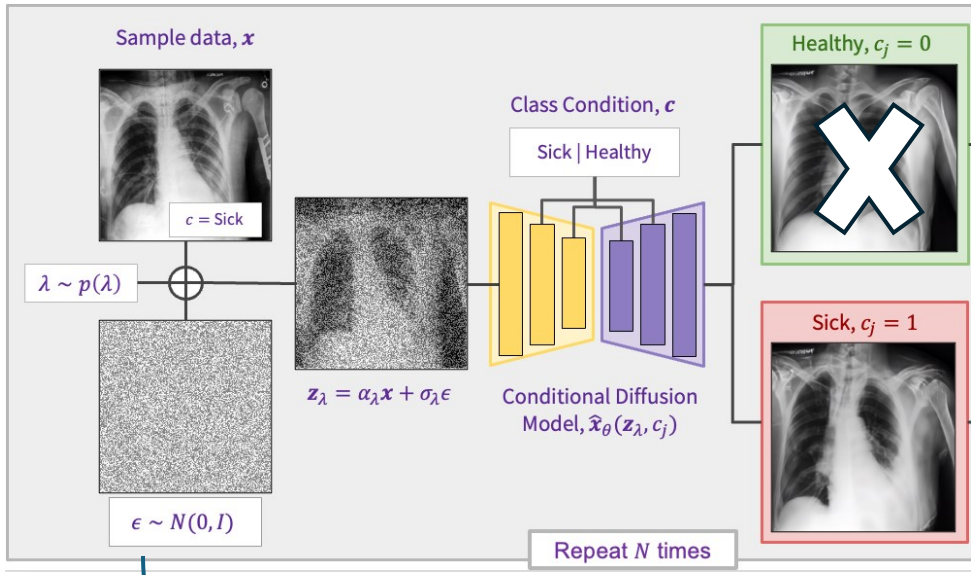
- *Conditional diffusion is different from time conditioning*
- *Time conditioning is mandatory and shared across all samples*
- *Conditional information is task-specific and steers generation toward a desired outcome*

CONDITION INJECTION

- The conditioning information c is first transformed into a learnable embedding
- This embedding represents the desired class, attribute, or textual guidance, injected directly into the denoising network
- It influences intermediate feature representations at multiple layers
- The model now predicts noise conditioned on external information:

$$\varepsilon_\theta(x_t, c)$$

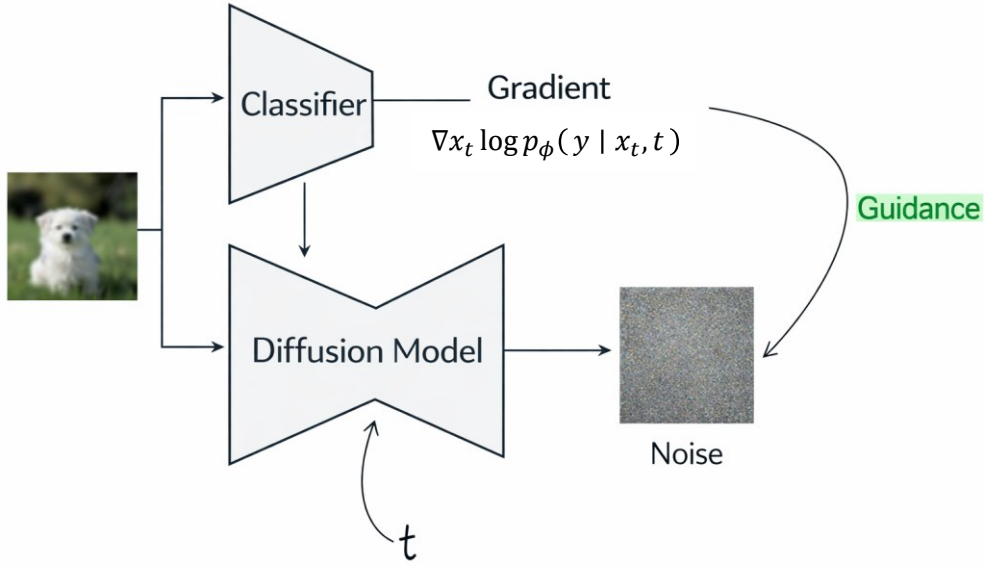
- The condition does not replace noise input; it modulates the denoising process, guiding it towards samples consistent with the condition
- At each step, the network removes noise in a way that aligns with the condition
- This ensures the final sample follows $p(x_0 \mid c)$, not the unconditional distribution



The model removes noise in a way that aligns with the given condition "sick"

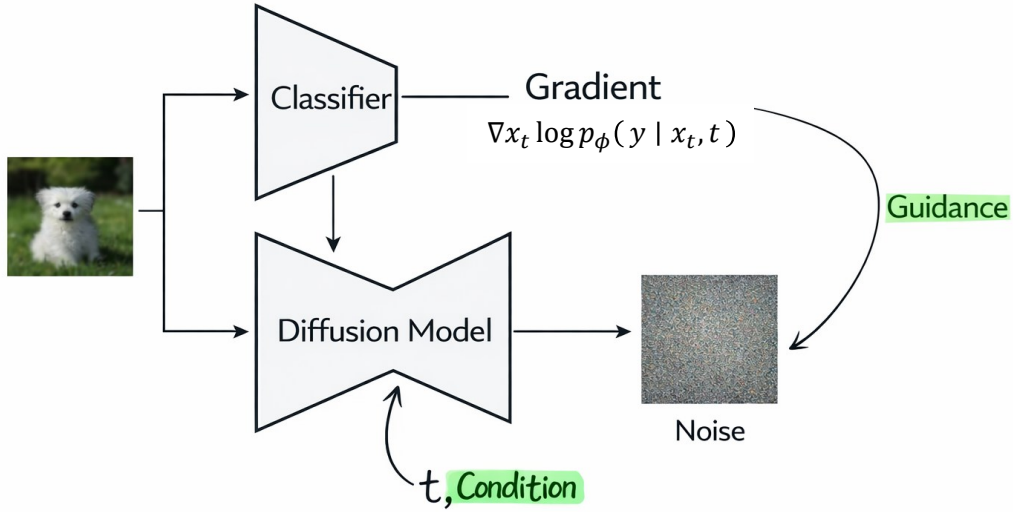
CLASSIFIER GUIDED DIFFUSION

- Guided diffusion does not condition the denoising network directly
- Instead, an external classifier provides guidance during sampling
- The classifier estimates how likely a noisy image matches the desired condition
- Its gradient is used to steer the denoising process toward that condition
- The denoising model remains unconditional
- Control is applied by modifying the reverse diffusion trajectory
- This produces samples that better align with the specified condition
- It gives stronger control over generated content and sharper alignment with the condition



CLASSIFIER-FREE GUIDANCE

- Classifier-free guidance removes the need for an external classifier
- The diffusion model is trained both with and without conditioning
- At sampling time, conditional and unconditional predictions are combined
- The difference between the two provides a guidance signal
- It leads to simpler and faster sampling and gives flexible control via guidance scale
- This is the dominant conditioning approach in modern diffusion models



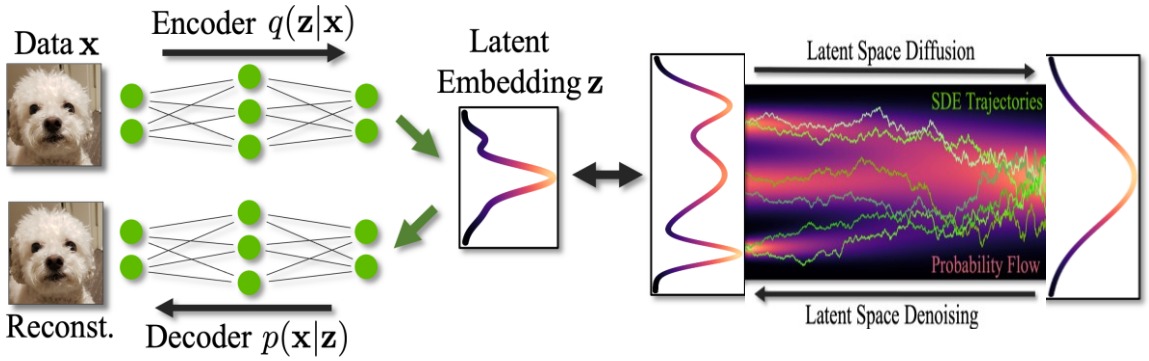
***Conditional diffusion**: condition enters the model*

***Guided diffusion**: condition steers the model*

***Classifier-free guidance**: the model guides itself*

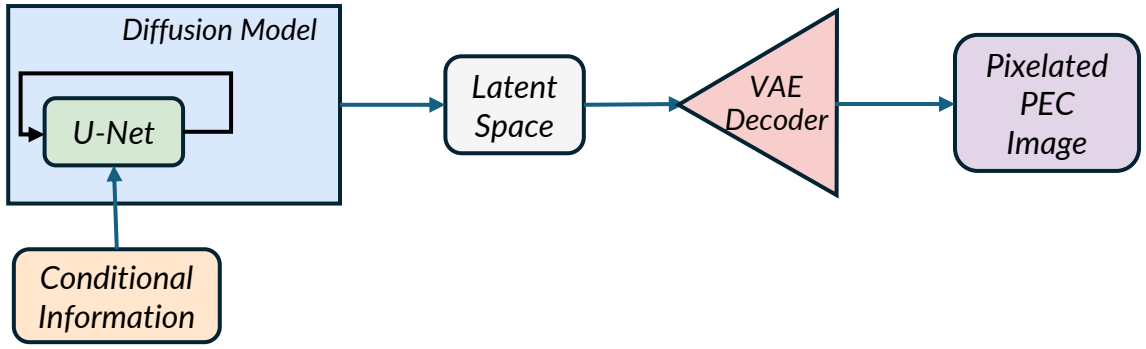
SCALING DIFFUSION MODELS

- We have seen the forward and reverse diffusion process in the pixel-space till now
- All the operations were directly transforming the pixel arrays
- However, pixel-space diffusion does not scale well
  - High-resolution images are high-dimensional
  - Memory and compute costs grow rapidly
  - Sampling becomes slow and expensive
- Scaling requires a more efficient representation: we can use latent space
- **Latent diffusion models** compress the images into a latent space and perform diffusion in the latent form
- Once the final output is generated, it is decoded back to pixel space



Latent Space Diffusion

LATENT DIFFUSION MODEL



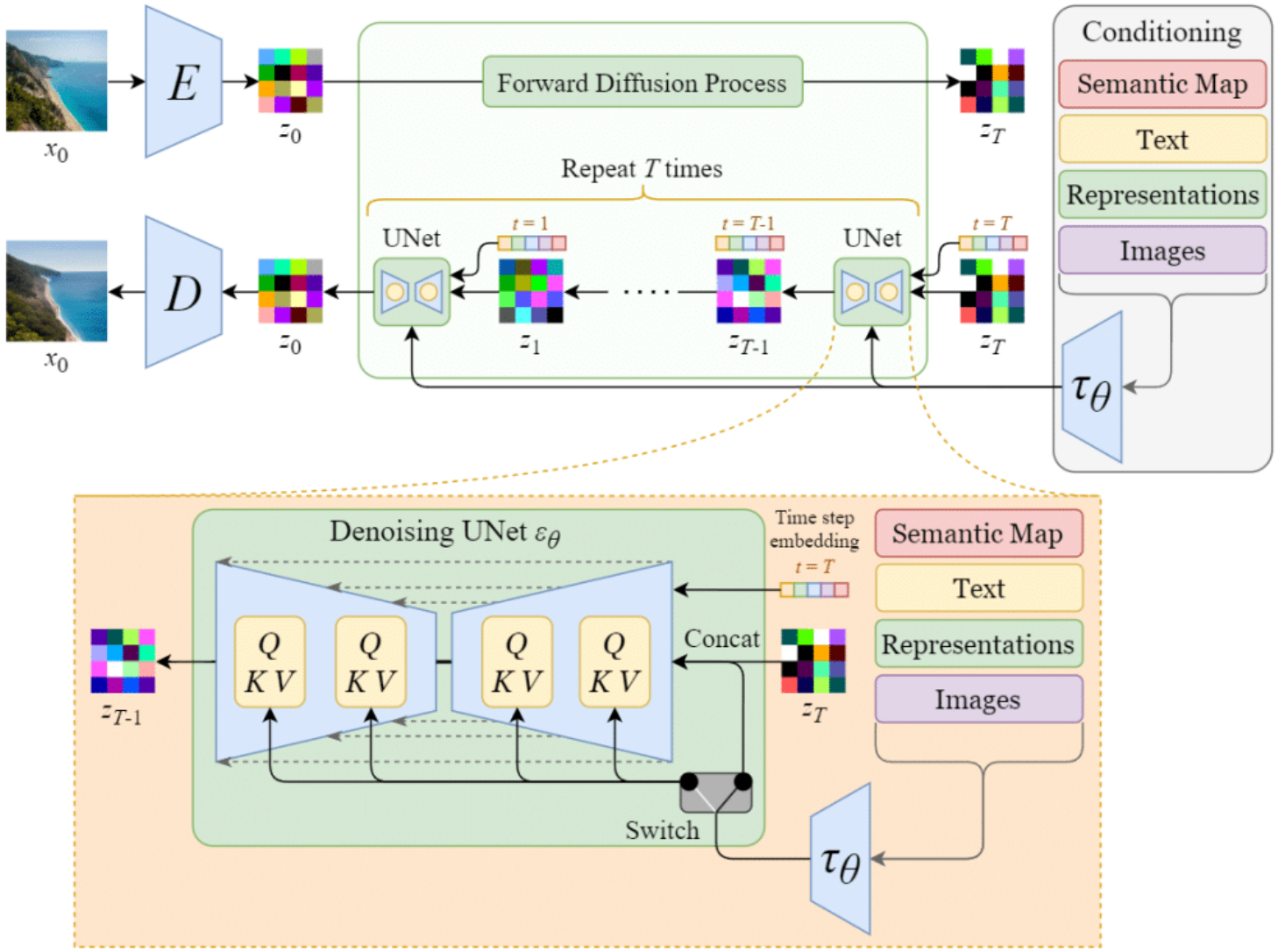
- Encoder maps image x to latent z, removing perceptual redundancy
- Diffusion Model (U-Net):
  - Operates entirely in latent space
  - Learns to denoise latent variables using the standard diffusion process
  - Predicts $\varepsilon_\theta(z_t, t, c)$
- Decoder maps the denoised latent z_0 back to image space

- Latent diffusion separates representation learning from generation
- A pretrained autoencoder compresses images into a low-dimensional latent space
- Diffusion in latent space drastically reduces computation and memory
- It enables efficient high-resolution image generation without sacrificing quality

STABLE DIFFUSION

- Stable Diffusion is an open-source latent diffusion model for large-scale text-to-image generation
- It combines a latent autoencoder, a diffusion model, and a text encoder
- Generation Process:
  - Start from Gaussian noise in latent space
  - Iteratively denoise using text-conditioned diffusion
  - Decode final latent to image space
- It provides high-quality text-to-image generation and efficient sampling at high resolutions
- Scales well to large datasets and complex prompts
- Stable Diffusion popularised diffusion models beyond research labs
- It powers many real-world applications in art, design, and content creation
- The model is open, extensible, and widely adopted in industry and academia

STABLE DIFFUSION ARCHITECTURE



- **Image Autoencoder**
  - Compresses images into a low-dimensional latent space
  - Diffusion is performed in this latent space for efficiency
- **Diffusion Model (U-Net)**
  - Learns to denoise latent representations step by step
  - Predicts $\varepsilon_\theta(z_t, t, c)$ conditioned on text
- **Text Encoder**
  - Converts text prompts into semantic embeddings
  - Text embeddings condition the denoising process via cross-attention